Kubernetes Cluster Deployment with Amazon EKS

An Enterprise-Grade, Cloud-Native Container Orchestration Platform

---

1. Project Title

Kubernetes Cluster Deployment with EKS: Automated Provisioning and Management of a Highly Available, Auto-Scaling Container Orchestration Platform on AWS

2. Abstract

Modern enterprises increasingly deploy applications as collections of loosely coupled microservices, but manually provisioning, networking, securing, and scaling the servers that host these microservices is complex, error-prone, and difficult to maintain at production scale. Traditional self-managed Kubernetes clusters require significant operational overhead to patch the control plane, manage etcd, configure high availability across zones, and integrate identity and access control, which increases both cost and risk of downtime. This project addresses these challenges by designing and deploying a production-ready Kubernetes cluster using Amazon Elastic Kubernetes Service (EKS), a fully managed Kubernetes control plane offered by AWS. The solution combines Infrastructure-as-Code (Terraform) to provision the Virtual Private Cloud, subnets, and managed node groups; AWS IAM Authenticator to map Kubernetes Role-Based Access Control (RBAC) to corporate identities; Amazon Elastic Container Registry (ECR) to securely store and serve container images; AWS Application/Network Load Balancers for intelligent traffic distribution; and the Kubernetes Cluster Autoscaler together with the Horizontal Pod Autoscaler to dynamically respond to workload demand. The purpose of the project is to build a reusable, automated, and secure blueprint for deploying resilient cloud-native applications with minimal manual intervention. The expected outcome is a highly available, self-healing, horizontally scalable Kubernetes environment spread across multiple availability zones that can absorb traffic spikes, roll out updates with zero downtime, and significantly reduce the operational burden traditionally associated with container orchestration.

3. Introduction

Container technology has transformed how software is built and shipped, allowing developers to package an application with all its dependencies into a single, portable unit. As the number of containers in an organization grows, however, manually tracking which container runs where, restarting failed instances, balancing load, and scaling capacity up or down becomes unmanageable. Kubernetes emerged as the industry-standard orchestrator to solve exactly this problem, but running Kubernetes "the hard way" on self-managed virtual machines still leaves teams responsible for the control plane's availability, security patching, and upgrade path. Amazon EKS removes this burden by offering Kubernetes as a managed service: AWS operates and scales the control plane across multiple availability zones while the user focuses on worker nodes and workloads. This project is important because it demonstrates, end-to-end, how an organization can move from manual, error-prone server clustering to a fully

automated, Infrastructure-as-Code driven deployment pipeline. It brings together several AWS-native services into a single continuous infrastructure loop, giving hands-on exposure to cloud architecture, identity and access management, container registries, and elastic scaling — all core skills for modern DevOps and cloud engineering roles.

4. Key Points of the Project

- Managed EKS control plane provisioned across multiple Availability Zones for high availability.
- Infrastructure-as-Code (Terraform) to automate creation of the VPC, subnets, security groups, and managed node groups.
- AWS IAM Authenticator integration to map Kubernetes RBAC roles to corporate/IAM identities for secure, auditable access.
- Amazon ECR as a private, encrypted container registry feeding images securely into cluster pods.
- AWS Load Balancer Controller for intelligent Layer 4/Layer 7 traffic routing to services and ingress resources.
- Cluster Autoscaler and Horizontal Pod Autoscaler to dynamically add/remove nodes and pods based on real-time demand.
- Rolling updates and self-healing deployments that achieve zero-downtime releases.
- Centralized logging and monitoring hooks (e.g., CloudWatch/Prometheus-ready) for cluster observability.

5. Advantages & Limitations

Advantages

- Reduced operational overhead since AWS manages, patches, and scales the Kubernetes control plane.
- High availability by default, with the control plane and worker nodes spread across multiple AZs.
- Elastic scalability that automatically matches infrastructure capacity to real traffic, optimizing cost.
- Strong security posture through native IAM integration, private ECR repositories, and fine-grained RBAC.
- Faster, safer deployments with zero-downtime rolling updates and self-healing pods.
- Repeatable, version-controlled infrastructure via Terraform, enabling consistent environments across dev/stage/prod.

Limitations

- AWS EKS incurs an hourly control-plane charge in addition to worker node and load balancer costs.
- Steeper learning curve, requiring familiarity with Kubernetes concepts, Terraform, and AWS IAM.

- Vendor lock-in risk, as some managed integrations are AWS-specific and not trivially portable to other clouds.
- Initial cluster and networking setup (VPC, subnets, security groups) can be complex to get right securely.
- Autoscaling reacts with some latency, so extremely sudden traffic spikes may briefly outpace new capacity.

6. Components in the Diagram

Terraform (IaC Layer): Defines and provisions the VPC, public/private subnets, security groups, and managed node groups declaratively, ensuring the entire infrastructure is version-controlled and repeatable.

Amazon EKS Control Plane: The fully managed Kubernetes control plane (API server, etcd, scheduler, controller manager) run by AWS across multiple Availability Zones for resilience.

Managed Worker Node Groups: EC2 (or Fargate) instances that register with the EKS control plane and actually run the application pods; scaled automatically by the Cluster Autoscaler.

AWS IAM Authenticator: Bridges AWS IAM identities with Kubernetes RBAC, allowing corporate users/roles to authenticate to the cluster with permissions mapped to Kubernetes namespaces and resources.

Amazon ECR (Elastic Container Registry): A private, encrypted Docker-compatible registry that stores built container images and securely serves them to the cluster's pods during deployment.

AWS Load Balancer (ALB/NLB): Distributes incoming application traffic across healthy pods, integrating with Kubernetes Ingress/Service resources for Layer 7 or Layer 4 routing.

Cluster Autoscaler / HPA: Monitors resource utilization and pending pods to automatically add or remove worker nodes (Cluster Autoscaler) and pod replicas (Horizontal Pod Autoscaler) in response to load.

Monitoring & Logging: Collects cluster and application metrics/logs (e.g., via CloudWatch or Prometheus/Grafana) to provide observability into cluster health and performance.

7. Future Scope

- Integrate a GitOps workflow (e.g., ArgoCD or Flux) for fully automated, Git-driven continuous deployment.
- Add a service mesh (e.g., Istio or AWS App Mesh) for advanced traffic management, mTLS, and observability.
- Implement multi-cluster / multi-region failover for disaster recovery and even lower-latency global access.
- Extend monitoring with a full Prometheus + Grafana stack and centralized alerting for proactive incident response.
- Introduce cost-optimization features such as Spot Instance node groups and Karpenter for smarter, faster autoscaling.

8. Conclusion

The Kubernetes Cluster Deployment with EKS project successfully demonstrates how a combination of Infrastructure-as-Code, managed AWS services, and native Kubernetes autoscaling can replace manual, error-prone server clustering with a resilient, automated, and secure container orchestration platform. By provisioning the control plane, networking, identity mapping, container registry, load balancing, and autoscaling as a single continuous

infrastructure loop, the project delivers a production-ready blueprint capable of achieving zero-downtime deployments and elastic scalability across multiple availability zones. Beyond the technical deliverable, the project provides practical, hands-on experience with core cloud-native and DevOps tools — Terraform, EKS, IAM, ECR, and Kubernetes autoscalers — that are directly applicable to real-world enterprise cloud engineering roles, making it both a functional platform and a valuable learning exercise in modern infrastructure automation.